The vibes that I summoned…

Thoughts on working with AI agents.

Chris Pahl · 2026

The Spectrum (on Reddit)

Full manual
every keystroke yours

Full vibecode
just make it work

⟵ more control · more understanding | more productivity · more delegation ⟶

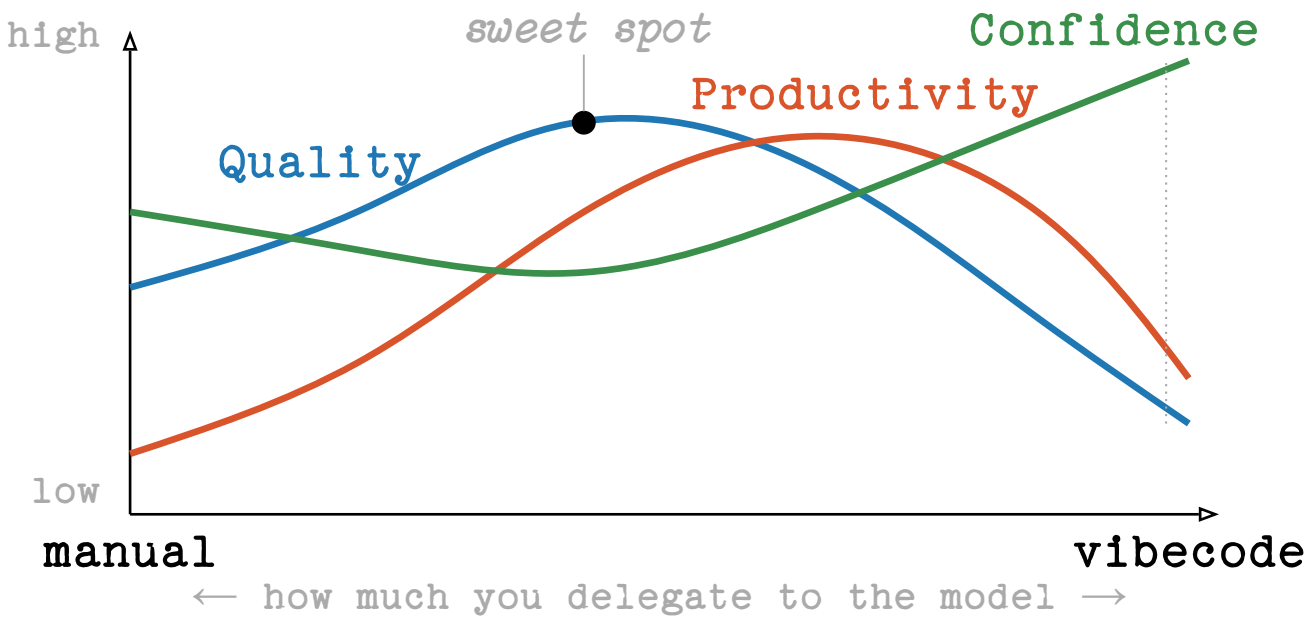
General tendency:

Using GenAI is a tradeoff between control and productivity.

Prompt:

“Imagine Donald Duck as realistic human. Remove the hat.”

My take: Quality vs Productivity



Quality peaks in the middle. Productivity follows it down. Confidence diverges from both -> Dunning-Kruger zone!

16%

But there's already data:

-19%

slower with AI

experienced devs · own mature repos · predicted +24%

METR · RCT · 2025

~40%

insecure programs

Copilot output across 89 security-relevant scenarios

NYU CCS · 2021

8×

more duplicated code

block-level clones up · refactoring 24% → 10%

GitClear · 211M LoC · 2024

-7.2%

delivery stability

drop with AI adoption · 39% distrust AI code

DORA / Google · 2024

29.5%

Python with CWEs

AI-generated code in real GitHub repos · 43 CWE categories

Fu et al. · ACM TOSEM · 2025

↓

trust = less scrutiny

more trust in GenAI ⟹ less critical thinking applied

Lee et al. · Microsoft + CMU · CHI 2025

Good use cases

rubber-ducking ideation

explain unfamiliar code test generation

pair programming boilerplate refactoring assist

learning accelerator documentation drafts naming things

summarisation regex / SQL crafting edge-case brainstorming

translation code review companion mock data / fixtures

error decoding search engine log analysis proofreading

automation

Risks for all

ecological cost

Convincing hallucinations

schools

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

communities thinning out

erosion of trust

Risks for us

ecological cost

Convincing hallucinations

schools

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

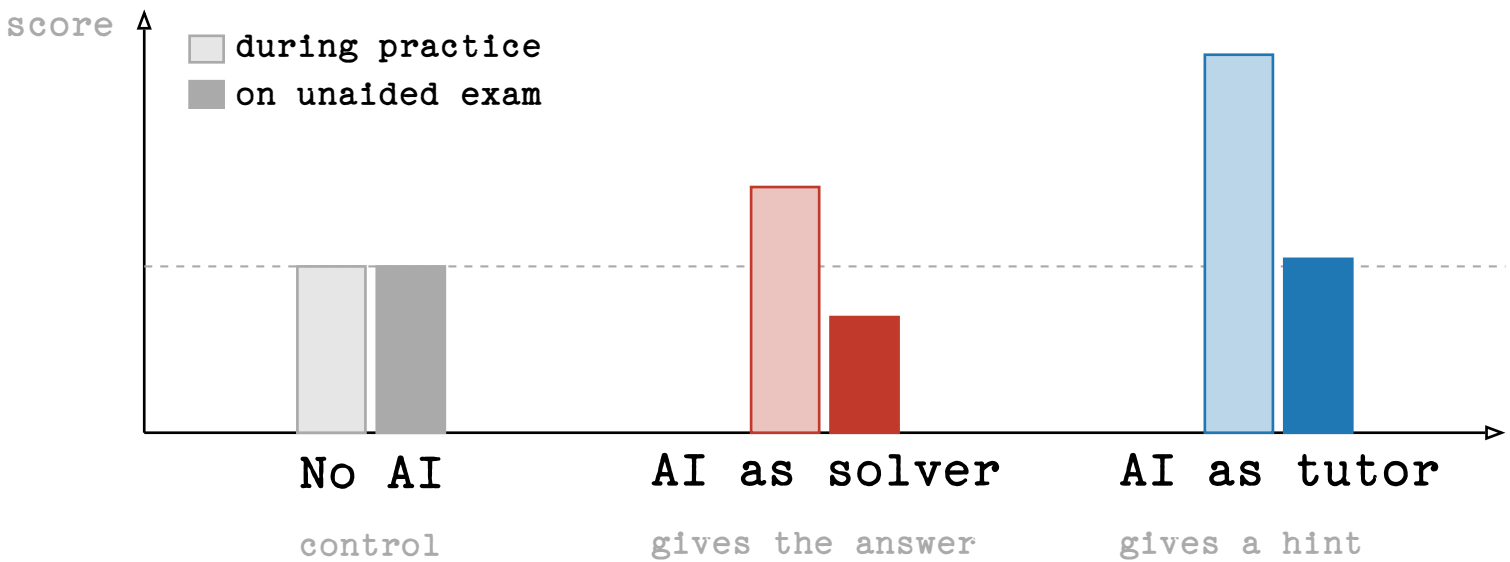
communities thinning out

erosion of trust

Exhibit A: schools.

Maybe it's not so much about the tool —
but about how we use it?

Same tool, opposite outcomes



Bastani et al., *Generative AI Can Harm Learning*, SSRN 2024

Are *you* team tutor or team solver?

Keeping up with Claude

Our team's Claude Code account · April 2026

98.7%

suggestions accepted as-is

1.3% rejected. Is that a review process?

27,757

lines accepted last month

≈ 925 LoC/day · careful review ≈ 100–200 LoC/hour

Reviewing 27,757 lines carefully ≈ 138 h.

Human cognitive biases

Automation bias

We accept machine suggestions more readily than human ones.

click accept · review later · maybe

Anchoring

The first suggestion shapes the solution space — even when it's wrong.

the model frames the problem before you do

Confirmation bias

We hear what we already believe.

the prompt contains the answer we want

Authority bias

Eloquent + fast + confident reads as expert.

fluency ≠ correctness

Dunning–Kruger

We mistake competence we observe for competence we have.

"this is what I would have written"

Illusion of explanatory depth

We think we understand — until asked to explain.

"yes, I read the diff" → bug

Statistical model biases

Sycophancy

It's easy to talk the model out of a correct answer.

echo chambers — but for code

Historical / representation bias

Trained on the web — heavily modern, English, Western.

defaults track what's common, not what's right

Attention bias

First and last things dominate; the middle evaporates.

rules buried in long context get ignored

Omission bias

Rare and novel answers get suppressed by common ones.

the model is bad at being weird

Hen & Egg

1. If you don’t know what good software looks like —
   how do you write the right prompt?

2. If you can’t understand what the model just generated —
   how do you verify it?

3. If you can’t code (anymore) —
   how can you understand the diffs?

4. If you do not know what you build inside-out —
   how do you learn new designs?

How do *you* keep up with Claude?

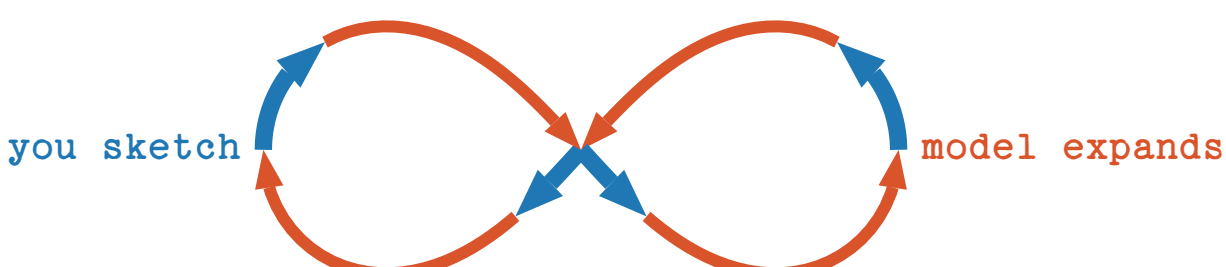
Five habits that helped me

1. Sketch first, generate then.
2. Split the work - keep some of it manual.
3. Have a real verification strategy.
4. Don’t make yourself replaceable.
5. Treat the AI like a colleague.

1. Sketch first, generate then

You set the anchor, you stay in the loop.

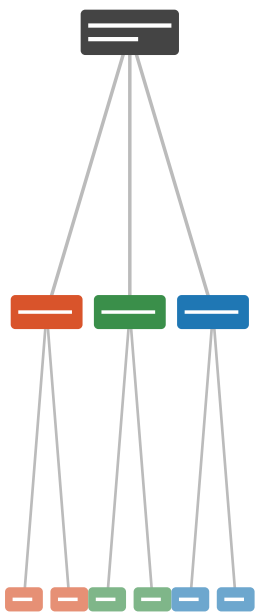
- Sketch the SQL query yourself,
  Then ask the model to review and optimise.
- Write the function signature and the docstring.
  Then let the model fill the body.
- Put TODO comments in your code
  Then let Claude work on them.



2. Split the work, do some manual

The parts you write are the parts you understand.

- Claude does not perform well, if given too big scopes.
  Split tasks up and prompt them individually.
- Claude can help you split up work.
  It just tends to work on the task right away.
- It is easy to lose understanding without noticing.
  Keep doing important things manually!
- Large diffs make it easy to get lost.
  Commit often so diffs stay reviewable.



3. Verify strategy before code

There are no shortcuts to quality.

Before you accept a generated change

name the signal that would tell you it’s wrong.

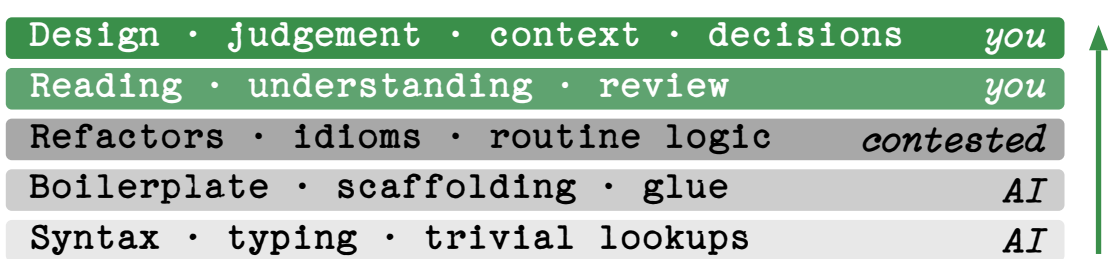
- Comparison against a reference implementation
- Property-based tests, fuzzing, …
- Type checkers, linters, static analysers
- **/review** and manual review by colleagues
- Replay against real production data
- Work actively on the code

Example: Noise library for Dart — I don’t speak Dart.

4. Don’t make yourself replaceable.

You only get replaced if you make yourself replaceable.

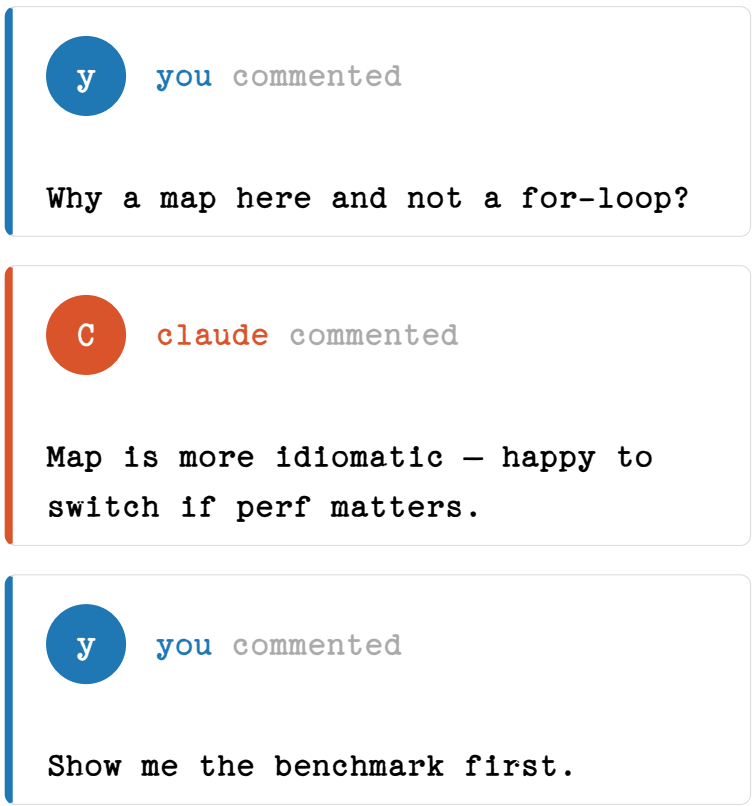
- Code was the source of truth before, that’s changing.
  Now it is context given via design documents.
- Spend the time Claude saves you on the parts it’s bad at.
  Namely: decisions, judgement, context, design.
- Reading code is much more important now than writing code.
  Keep your tools sharp. Be able to work without Claude.



5. Treat Claude like a colleague

Talk to it like your seat neighbor.

- **Explain the tasks at hand like you would to a junior colleague.**
  A very eager, junior colleague with seemingly infinite capacity.
- **Push back. Ask for alternatives. Let it explain. Disagree.**
  If you'd reject a colleague's PR for that reasoning, reject the model's.



Bonus: Programming as Theory Building

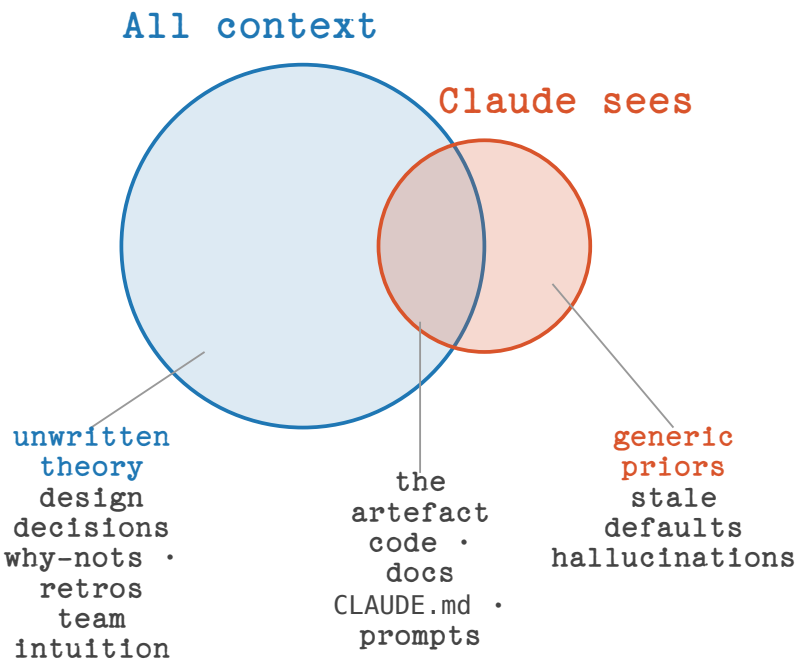
Peter Naur, 1985

- The code is a by-product. What you're really building is the theory — your team's shared mental model of the problem.
- Documentation captures the artefact, not the theory. The bugs, the dead ends, the "why not this?" decisions live in people's heads.
- This is why handovers fail. The theory doesn't transfer cleanly.

Peter Naur, 1928–2016
Photo: Wikimedia Commons

Bonus: What that means for Claude

- The model has no theory — it sees the artefact and fills the rest in.
  Confidently. It won't tell you which is which.
- Bad prompt: "Write tests for this file"
  It freezes today's behaviour, bugs included. Again, no mention.
- Your job is still to build the theory.
  The AI helps you write the code that follows from it.



Questions or Disagreements?

Homework:

PROGRAMMING AS THEORY BUILDING.